

Policy Gradient-Based Core Placement Optimization for Multichip Many-Core Systems

Wooshik Myung, Donghyun Lee^{ID}, Chenhang Song^{ID}, Guanrui Wang, and Cheng Ma^{ID}

Abstract—As many deep neural network models become deeper and more complex, processing devices with stronger computing performance and communication capability are required. Following this trend, the dependence on multichip many-core systems that have high parallelism and reasonable transmission costs is on the rise. In this work, in order to improve routing performance of the system, such as routing runtime and power consumption, we propose a reinforcement learning (RL)-based core placement optimization approach, considering application constraints, such as deadlock caused by multicast paths. We leverage the capability of deep RL from indirect supervision as a direct nonlinear optimizer, and the parameters of the policy network are updated by proximal policy optimization. We treat the routing topology as a network graph, so we utilize a graph convolutional network to embed the features into the policy network. One step size environment is designed, so all cores are placed simultaneously. To handle large dimensional action space, we use continuous values matching with the number of cores as the output of the policy network and discretize them again for obtaining the new placement. For multichip system mapping, we developed a community detection algorithm. We use several datasets of multilayer perceptron and convolutional neural networks to evaluate our agent. We compare the optimal results obtained by our agent with other baselines under different multicast conditions. Our approach achieves a significant reduction of routing runtime, communication cost, and average traffic load, along with deadlock-free performance for inner chip data transmission. The traffic of interchip routing is also significantly reduced after integrating the community detection algorithm to our agent.

Index Terms—Community detection algorithm, deadlock-free placement, physical mapping, reinforcement learning (RL).

I. INTRODUCTION

IN RECENT years, many algorithms based on deep neural networks have made remarkable progress in many application fields, such as computer vision [1], [2], speech recognition [3], playing games [4]–[6], robotics [7], natural language

processing (NLP) [8], automation [9], and others [10]. Furthermore, various programming frameworks [11], [12] for deep learning promote the use in a wider range of high-performance tasks. With the rapid development of algorithms, networks with deeper layers and heavier calculations are constantly being designed. Most of the neural network algorithms are typically performed on devices, such as central processing unit (CPU) and graphics processing unit (GPU). Depending on the complexity of applications, application-specific integrated circuit (ASIC), such as TPU [13], DNPU [14], and Eyeriss [15], are designed to accelerate deep neural network training. They are much faster and consume less power than CPU and GPU. In other hand, neuromorphic computing devices consisting of multichip many-core systems, such as TrueNorth [16], SpiNNaker [17], Loihi [18], NeuroGrid [19], Simba [20], and TIANJIC [21], are attracting attention as a next-generation deep neural network architecture, which has many memories and units for parallel computing. The distributed memory and closely integrated computation cores provide high speed and low power consumption.

After designing a neuromorphic device, two steps, which are shown as Fig. 1, should be taken to actually use it: logical mapping and physical mapping, respectively. In the logical mapping stage, the computational graph of the neural network is partitioned into several groups with specific size [22], which is generally represented as a directed acyclic graph (DAG), considering the computational capability of each node on physical communication devices. This graph is not yet assigned on a physical device, so we refer to them as a logic graph. In the physical mapping stage, we place these cores of the logic graph into physical devices, considering the topology, routing strategy, and buffer size of them. However, the typical characteristics of neuromorphic processors, such as decentralization, local storage, and many-core parallel computing, make physical mapping more difficult and complex. In addition, problems such as deadlock and higher latency, caused by multicast routing used to solve the insufficient fan-in and fan-out capability of a single core, should also be considered. For a multichip system, we should also take the difference in communication capabilities of inner chip and interchip routings into account. The data transfer rates on interchip ports, such as LVDS [23] and SERDES [24], are generally much slower than those on inner chips. Therefore, it is advantageous to improve routing performance when nodes with strong connectivity are placed on the same chip.

Manuscript received 14 February 2021; revised 8 August 2021; accepted 26 September 2021. Date of publication 13 October 2021; date of current version 4 August 2023. This work was supported by the National Key Research and Development Program of China under Grant 2018YFE0200200. (Corresponding author: Cheng Ma.)

The authors are with the Center for Brain-Inspired Computing Research, Department of Precision Instrument, Tsinghua University, Beijing 100084, China (e-mail: mingys18@mails.tsinghua.edu.cn; lidx18@mails.tsinghua.edu.cn; sch20@mails.tsinghua.edu.cn; wgr16@mails.tsinghua.edu.cn; macheng@tsinghua.edu.cn).

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/TNNLS.2021.3117878>.

Digital Object Identifier 10.1109/TNNLS.2021.3117878

2162-237X © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

Core placement optimization is an NP-hard problem [25], and the positional relationship of each core would greatly affect the performance of a neuromorphic system. Although some placements for a logic graph have similar power consumptions, their latencies are totally different. The tradeoff between them is the key to find the optimal core placement.

In many recent studies [26]–[31], reinforcement learning (RL)-based approaches proved their superiority to solve various combinatorial optimization problems. To handle the multiobjective optimization problems with complex constraints, we propose an RL-based core placement optimization method both for single-chip and multichip systems. We leverage the capability of deep RL (DRL) from indirect supervision as a direct nonlinear optimizer. To deal with the tradeoff between power consumption and latency, we use the approximation function for the former one as the objective function and find a low-latency placement from the optimized results obtained by the already trained neural networks. The parameters of our policy network are updated by proximal policy optimization (PPO) [32]. For the multichip physical mapping, we use the community detection algorithm to reduce the difficulty of large-scale core placement optimization. We use TIANJIC [21] array as our objective hardware, forming a 2-D mesh Network-on-Chip (NoC) topology. Each TIANJIC chip is composed of 256 cores, which is the basic computing unit and realizes the functions of axon, synapse, dendrite, soma, and router. Neural networks of different sizes, including multilayer perceptron (MLP), convolutional neural network (CNN), and the workload of ResNet50 [33], are used as optimization datasets. We evaluate our RL-based agent for mapping in a single chip (with the array size of 8×8 and 16×16) for four small datasets (50–150 cores) and also evaluate placement performance with bigger size of datasets, over 600 cores and 1400 cores on three and six chips, respectively. The optimized results of the proposed approach are compared with other baselines, such as two sequential placement schemes, the random search, and simulated annealing (SA). The C++-based routing cycle-accurate simulator is used to calculate routing performance, such as latency and power consumption. The specific contributions of this work are given as follows.

- 1) We take core placement optimization as an RL problem and formulate the model-to-core mapping to improve the routing performance. In addition, we present an approach that considers routing problems as a soft constraint using exponential when designing reward function. We mainly focus on finding a deadlock-free placement, and the experiment results show that it is effective.
- 2) We use a simple version of DRL that has only a single step size of every episode and employ PPO to update the parameters of our policy network. In addition, a policy network consisting of two small-size fully connected layers to output probability distributions is utilized. These simplified conditions accelerate the whole training time and reduce computation resources.
- 3) We evaluate our RL-based agent on multiple datasets with various sizes of fully connected layers and convolutional layers for a single-chip system. The experiment

results show that our agent reduces 63.3%, 66.9%, 27.4%, and 19.4% latency and also decreases 50.5%, 48.1%, 67.2%, and 10.9% power consumption on geometric average compared with two sequential placement schemes, the random search, and SA, respectively.

- 4) We apply the community detection algorithm as a partitioner of a network graph for a multichip system, and it is performed once before training. Two datasets of large size convolutional layers and the workload of ResNet50 are used to evaluate our approach in a multichip system. The results are compared to the placements generated by manual and our agent with weighted reward function for differentiating between inner chip and interchip routings. It shows that our approach, including the customized community detection algorithm, can effectively reduce the traffic of interchip routing and also reduce the entire training time.

The rest of this article is organized as follows. In Section II, we briefly discuss related studies of physical mapping. In Section III, some preliminary definitions of mapping are described, and a constrained optimization problem is also formulated. In Section IV, we present our RL-based physical mapping optimizer. A detailed description of the experimental setup and comparison with other mapping algorithms has been given in Section V. Finally, the conclusion that gives a brief summary is drawn in Section VI.

II. RELATED WORK

A. Application Mapping for Generic NoC

Application mapping for NoC is acknowledged to be an NP-hard problem. The major objective of this study is to find an optimal arrangement of many-core systems with keeping in mind its constraints, such as latency, throughput, bandwidth, and power consumption. Application mapping algorithms for NoC can be grouped into two categories: static and dynamic mapping. Static mapping indicates that all applications are mapped in the design phase, and they are not altered after mapping, whereas the latter represents a mapping process that maps applications to NoC, which is executed in real time. In this article, we only handle static mapping. Static mapping algorithms generally can be classified into two branches: mathematical and searched-based algorithms. For mathematical methods, Bender [34] presented a mixed-integer linear program (MILP)-based mapping technique for heterogeneous systems. Rhee *et al.* [35] proposed an MILP-based formulation for an application mapping technique of NoC, and they stated that energy consumption and congestion are improved. In [36], the integer linear program (ILP)-based method that maps applications to complex multiprocessor and multithreaded network processors was presented. However, the main bottleneck of these methods is the execution time. As the number of cores integrated into a single chip increases, the search-based mapping method is more focused recently. There are various heuristic search-based mapping solutions, such as genetic algorithm (GA) [37], [38], ant colony optimization (ACO) [39], [40], particle swarm optimization (PSO) [41], [42], and SA [43], [44]. Srinivasan and De Micheli [45]

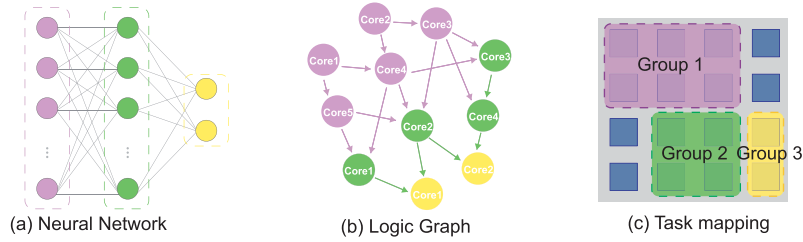


Fig. 1. Model-to-core mapping process: (a) original neural network model; (b) original neural network is partitioned and is represented as a weighted DAG; and (c) logic cores are one-to-one mapped into physical nodes.

proposed a mapping algorithm, called NMAP, that maps the cores onto NoC with mesh topology to minimize the average communication delay. It has three phases and is performed under bandwidth constraints. Janidarmian *et al.* [46] proposed one of the constructive heuristic algorithms, called Onyx, with less complex bandwidth constraint. A detailed survey of application mapping schemes and performance comparison for NoC can be found in [47] and [48], but these previous studies have only been carried out for a single-chip many-core system. Compared with single-chip many-core systems, multichip systems face two additional challenges: communication delay caused by unbalanced inner chip and interchip communication bandwidth; with the increase in the number of chips, the search space becomes much more spacious, which further demands the performance of placement optimization algorithm.

B. Physical Mapping for NN Chip

In the process of mapping high-level neural network models to physical communication devices, such as a neuromorphic chip, there are two steps, called logical and physical mapping. However, current studies focus more on the former one, i.e., fully folded [49], fully unfolded [22], or semifolded [50]. These studies are not considering the optimization that can be obtained in the physical mapping process. On the TrueNorth platform, Akopyan *et al.* [16] formulated the mapping neuromorphic cores to minimize spike communication power as a minimizing wire-length problem in VLSI placement. They used four different global placement algorithms to perform minimization and used the placements with minimum wire length as the actual implementation. IBM CPLACE [51], which is an analytical placer to design high-performance server microprocessors, gaming chips, and other ASICs, was also used to perform minimization. On the SpiNNaker platform, Jin [52] presented a linear mapping scheme, that is, neurons are uniformly allocated to processors in order. On the Eyeriss platform, Chen *et al.* [15] proposed the row stationary (RS) dataflow scheme that maps the computation of any given CNN models onto the physical cores array of an accelerator. As their statement, this method optimizes energy efficiency by reconfiguring a form of computational graph to reuse data as much as possible locally. On the TIANJIC platform, Wu *et al.* [53] proposed an RL-based scheme to improve the performance of multichip many-core neural network systems. The work related to core placement aims to optimize the latency of throughput of a multichip many-core system for neural network training, and we further

consider the deadlock problem caused by multicast routing in this work, which is not mentioned in previous works.

C. RL Approach for Placement Optimization

Recently, many studies have applied machine learning methods to combinatorial optimization problems and outperform traditional methods by a large margin [54]–[58]. Since RL is quite appropriate to solve decision-making problems, the RL-based method have achieved remarkable results [28]–[30]. Mazyavkina *et al.* [31] synthesized recently proposed RL-based approaches and made a comparison with traditional algorithms. Placement optimization is one of the combinatorial optimization problems, and it is a high-complexity and time-consuming step of the chip design process. It is a challenging mission because of several factors, such as the size and topology of the input graph, properties of each node, many requirements, and constraints. Lately, there are recent studies [59]–[64] trying to solve this problem with RL. Vashisht *et al.* [59] proposed a method that is combining RL with heuristics algorithm, SA. In this work, RL was used to provide better initialization for SA, and their result showed that it can improve the final placement performance. Mirhoseini *et al.* [62] proposed an RL-based approach to find the optimal placement for integrated circuits. Macros are placed by a PPO agent, and they used force-directed algorithm [65] to place standard cells. They also enabled policy networks to generalize to unseen blocks and transfer learning. Aside from the employment in the phase of chip design, placement optimization algorithms are also useful in improving training time when implementing machine learning on multiple conventional computation devices, such as CPU and GPU. The computational requirements for training neural networks are constantly increasing in the last couple of years. Thus, many machine learning practitioners use distributed training environments [66], which means training a neural network on multiple computation devices. Device placement can be framed as a problem of learning to partition computation graphs and map to available devices. The objective of this task is to find a match of operations to devices and minimize training time. Mirhoseini *et al.* [68] proposed REINFORCE [67]-based, which is one of the policy gradient algorithms, hierarchical method consisting of grouper and placer for device placement. The authors used a sequence-to-sequence recurrent neural network (RNN) as a policy network to predict the placement. This approach requires human experts to manually partition operations as a preprocessing

and is also limited to only small graphs. Later, the authors promoted a more flexible approach, performing with tens of thousands of operations with automatically grouping [69]. Gao *et al.* [70] proposed Spotlight, which allows them to find better placement in spite of consuming less time. They also proposed Post [71], which integrates cross-entropy minimization and PPO to attain theoretically guaranteed optimal efficiency. Addanki *et al.* [72] proposed Placeto, which uses graph embeddings to capture relevant information about the structure of the computational graph and train efficiently and generalize to new unseen graphs. In recent literature, Jeon *et al.* [73] proposed the Baechi system, which is an algorithmic solution for a small cluster of memory-constrained devices.

III. PROBLEM STATEMENT

A. Physical Mapping

Physical mapping is a process of mapping logic graphs to the physic cores of a communication device. The application task graph, consisting of some cores and directional weighted edges, is to be scheduled on a physical NoC. With this in mind, we present some definitions first.

Definition 1: An application task graph is represented as a weighted DAG, denoted as $G(A, E)$, where $a_i \in A$ represents the set of application tasks and $e_{i,j} \in E$ represents the set of directional weighted edges between the applications a_i and a_j . The communication weight of them is denoted as $\text{comm}_{i,j}$.

Definition 2: The NoC topology graph is a directed graph $T(N, P)$, where each vertex $n_i \in N$ represents a node of the device that connects to a local router, and each path $p_{i,j} \in P$ represents the shortest communication path from n_i to n_j under dimension order routing strategy. All nodes have links $l_{i,j}$ with their adjacent vertices based on the mesh structure.

Definition 3: A mapping function of application task graph $G(A, E)$ to the topology graph $T(N, P)$ is defined as follows [43]:

$$\begin{aligned} \text{map} : G \rightarrow T \quad \text{s.t.} \quad \text{map}(a_i) = n_i \\ \forall a_i \in A, \quad \exists n_i \in N \\ |A| \leq |N| \end{aligned} \quad (1)$$

where $|A|$ and $|N|$ represent the number of application tasks and routers of physical device.

In NoC designs, application mapping determines the topological arrangements to optimize certain metrics of performance, such as latency, throughput, and power. The results of placements are generally evaluated by various NoC simulators, such as BookSim [74], Noxim [75], and DARSIM [76]. Also, several mathematical models to calculate performance constraints, such as communication cost, latency, throughput, and power consumption, are presented in [48].

The average latency of NoC can be represented as follows [48]:

$$\text{Latency} = \frac{1}{|N|} \sum_{i=1}^{|N|} \frac{1}{N_i} \sum_{j=1}^{|N|} \text{Latency}_{i,j} \quad (2)$$

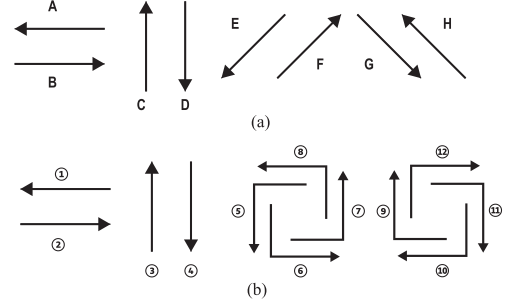


Fig. 2. (a) Eight connection relationships and (b) 12 shortest paths in the 2-D mesh topology.

where N_i is the number of packets received by a single processor after i warm up time and $\text{Latency}_{i,j}$ is the latency of packet j .

The communication cost is calculated using the following equation [77]:

$$\text{Communication Cost} = \sum_{\forall v_i, v_j} \text{comm}_{i,j} \times N_{i,j} \quad (3)$$

where $N_{i,j}$ is the Manhattan distance, which is calculated by $|x_i - x_j| + |y_i - y_j|$. In the mesh topology, it is the number of the minimum hop distance of path $p_{i,j}$ between two nodes n_i and n_j .

In [21], the average power consumption is presented in following equation:

$$\text{Power} = \frac{\text{Communication Cost} \times E_{|h|}}{t} \quad (4)$$

where $E_{|h|}$ is the energy of transmitting a packet for single hop distance.

In our case, since the working period of cores is fixed, t is not considered, and the average power is perfectly linear with communication cost.

These mathematical models demonstrated above are functional in practice to evaluate the routing performance of NoC systems. However, they are not suitable to be used as objective functions of our RL-based approach because they do not take into account the deadlock problem. Based on the above definitions and formulas, Section IV presents a customized communication cost to meet our requisite. In the following, we introduce some background knowledge necessary for the core placement problem of our system to optimize routing performance.

B. Routing Strategy

Once we have a particular topology and routing table for our system, deciding a routing strategy is the next step. Routing strategy defines a path of data transmission from the source router s to the destination router d in a particular topology. The shortest path $p_{i,j}$ from n_i to n_j represents a route between the two nodes with the minimum number of hops. In a 2-D mesh topology, there are eight connection relationships and 12 shortest paths between any two nodes, as shown in Fig. 2.

In our device, there are particular operative nodes, called multicast nodes, which distribute efficiently packets to other nodes. In general, a router of physical communication devices

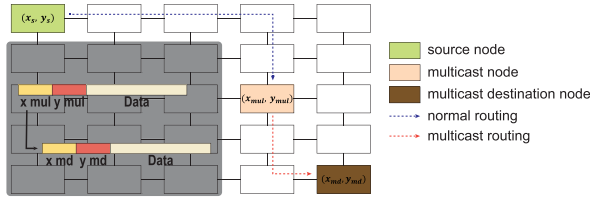


Fig. 3. Example of multicast routing. A packet is generated in the source node (x_s, y_s) and routed to its destination node (x_{mul}, y_{mul}) . If the destination node is a multicast node, it copies the data of the received packet and reroutes the packet to the multicast destination node (x_{md}, y_{md}) . Finally, it is the same with the source node s sending to multiple destination nodes d_{mul} and d_{md} , with only one packet generation.

has not enough fan-out to meet that of neurons in the neural network. Therefore, multicast routing is adopted in order to expand the maximum number of outputs of the router. An example of multicast routing is shown in Fig. 3. Unlike point-to-point routing, which terminates the process when a packet is delivered from the source node to the destination node, the received data are copied, and the packet is rerouted to the previously built-in multicast destination node simultaneously when it is transmitted to a multicast node. Consequentially, the packet can be distributed from one core to multiple cores in only one packet generation. However, this data transmission scheme causes the deadlock problem in spite of using dimension order routing, one of the well-known deadlock-free routing algorithms.

C. Routing Problems

There are several typical problems of the data transmission process on NoC, such as deadlock, livelock, failures of links or nodes, and load unbalancing. They definitely degrade the routing performance of communication devices. A packet transmitted from a specific router may not reach its assigned destination infinitely because of these reasons during routing; thus, the inability to deliver packets should be considered when we map logic cores to our physical device. In this work, we mainly focus on the deadlock problem. Deadlock, one of the causes of poor system performance, refers to a situation in which a group of routers is requesting resources from each other, resulting in an infinite wait, and cannot make progress anymore. Fig. 4(a) shows an example when a deadlock occurs in a 2-D mesh topology. Directed dependence of four nodes forms a cycle, and they are waiting on each other to release the resource.

We can avoid the deadlock problem in the process of designing NoC architecture as follows.

- 1) Allow the preemption of packets that may be deadlocked. They can be discarded and retransmitted, but this solution can lead to high latency and low efficiency of routing.
- 2) Use enough buffer size. However, this is directly related to system size, which may cause unnecessary waste of resources.
- 3) Use deterministic routing strategy. Dimension order routing is the most commonly used and famous deadlock-free routing algorithm, and it is to ensure that

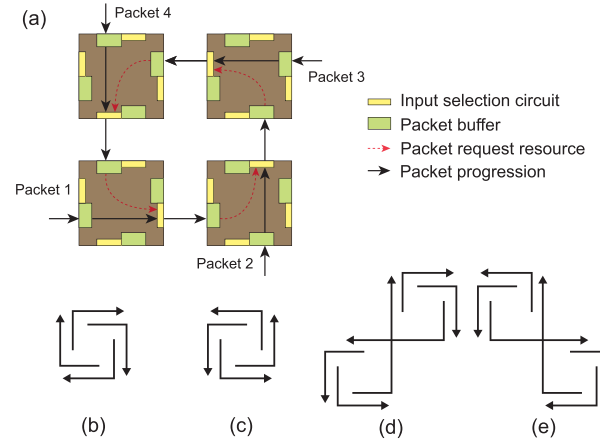


Fig. 4. (a) Example of deadlock. The packets of four routers are waiting for each other to release their resources; thus, they cannot make progress anymore, and the destination nodes cannot receive them infinitely. (b)–(e) Examples of directional dependence cycles that can occur in a 2-D mesh structure.

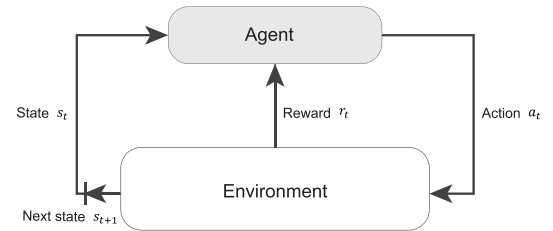


Fig. 5. Typical process of RL. The agent is trained to get the higher rewards in a single episode through interactions with its environment.

some routing directions that formed dependence cycles [⑥, ⑧, ⑩, and ⑫ described in Fig. 2(b)] are avoided.

However, the two former methods can be inefficient because of the reasons mentioned above. About the last solution, applying multicast routing can still lead to deadlock problems because all connection relationships forming the dependence cycles shown in Fig. 4(b)–(e) can be emerged. We explain in detail how to deal with this problem in Section IV.

D. Reinforcement Learning

RL is a cut-and-try approach that learns through interaction with the environment based on the Markov decision process (MDP) [78], as shown in Fig. 5. At a specific time step t , the agent learning optimal actions receives a state s_t of state space $\mathcal{S}$ and determines action a_t , which belongs to the action space $\mathcal{A}$ according to its policy π . Environment returns the next state s_{t+1} , reward r_t , and state transition probability $P(s_{t+1}|s_t, a_t)$. Agent continues to select an action from timestep t to the end of the current episode and learns a policy to maximize total rewards obtained from a single episode. Agent estimates expected future rewards from timestep t through value function that represents how good it is for the agent according to the current policy. The state-value function that expects the current state value $V_\pi(s)$ [79] is defined as

$$V_\pi(s) = E_\pi[R_t | S_t = s] \quad (5)$$

and the state-action value function $Q(s, a)$ that expects the value of action a in state s is represented by

$$Q_\pi(s, a) = E_\pi[r_t | S_t = s, A_t = a]. \quad (6)$$

There are two types of learning methods to maximize the expected episodic rewards: value-based and policy-based. The value-based method is a way of predicting values for as many states or state-action pairs as possible using a tabular search or function approximation with neural networks, and Q-learning [80] is a representative value-based method. The policy-based method [81] parameterizes the policy and updates the parameter by performing gradient ascent. This process is represented by

$$\mathcal{J}_\theta = E_{\tau \sim \pi_\theta(\tau)} = \int \pi_\theta(\tau) r(\tau) d\tau \quad (7)$$

$$\begin{aligned} \nabla \mathcal{J}_\theta &= \int \nabla \pi_\theta(\tau) r(\tau) d\tau \\ &= \int \pi_\theta(\tau) \nabla \log \pi_\theta(\tau) r(\tau) d\tau \\ &= E_{\nabla \log \pi_\theta(\tau) r(\tau)} \end{aligned} \quad (8)$$

where $\mathcal{J}_\theta$ is an objective function which is expected episodic rewards and τ is represented for the episodic trajectory composed of state-action pair (s_t, a_t) .

The representative method of the policy-based method is REINFORCE [67], and the parameters of policy are updated in the direction of $\nabla_\theta \log \pi_\theta(\tau) Q_\pi(s_t, a_t)$. RL is a good approach to handle optimization problems with large search space. We propose an RL-based physical mapping method that uses the PPO algorithm to update policy network parameters only with a triplet — initial state, action, return of environment.

E. Community Detection

In order to expand the computation capability and memories of a single chip, a multichip system is usually employed. However, inner chip and interchip data transmissions do not have the same bandwidths, so they should be considered differently. Community detection algorithms typically cost less time than other heuristic optimization algorithms and, like our logic graph, can effectively analyze and detect clusters of networks. Community detection is the process of deriving some insight through the relationship between nodes and edges of a network graph, and it facilitates analysis by grouping nodes with high connection density. It is similar to the process of unsupervised clustering because it is performed without any prior knowledge of clusters. We generally determine the quality of the partitions with the modularity shown in the following equation [82]:

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{i,j} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \quad (9)$$

where m is the sum of weights of all edges, $A_{i,j}$ is the weight of the edge between nodes i and j , k_i is the sum of the weights of the edges connected to node i , and $\delta(c_i, c_j)$ is the function whether two nodes i and j are in the same community, which returns 1 if they are same and 0 otherwise.

The modularity of a partitioned graph is a scalar value between -1 and 1 , which is obtained by comparing the density of links inside and between communities. There are two types of the algorithm for detecting community: divisive algorithms [83]–[85] and agglomerative algorithms [86], [87]. The former initially considers the entire network as a community, and detecting and removing links from them are performed iteratively. The latter maximizes the objective function by recursively merging similar nodes or communities since each object is considered as a cluster in the beginning phase. In our case, we use the faster unfolding algorithm proposed by Tragg [87], which belongs to the latter type and guarantees high speeds. It is one of the hierarchical clustering methods to detect communities of network graphs taking into account the weights between nodes. In the initialization of this algorithm, all nodes are each considered as a community, and it performs the following two phases repeatedly. In the first phase, each community i tries to assign itself to the randomly selected neighbor community C_j and calculate the gain of modularity ΔQ shown as follows:

$$\begin{aligned} \Delta Q &= \left[\frac{\sum_{in} + k_{i,in}}{2m} - \left(\frac{\sum_{tot} + k_i}{2m} \right)^2 \right] \\ &\quad - \left[\frac{\sum_{in}}{2m} - \left(\frac{\sum_{tot}}{2m} \right)^2 - \left(\frac{k_i}{2m} \right)^2 \right] \end{aligned} \quad (10)$$

where $\sum_{in}$ is sum of the weights of the edges inside C , $k_{i,in}$ is the sum of the weights of the edges from i to nodes in C , and $\sum_{tot}$ is the sum of weights of the edges incident to nodes in C .

If ΔQ is positive and the community index i is bigger than j , community i is relocated to the community C_j , and it is performed for every community. In the second phase, a new network whose nodes are the communities found in the first step is rebuilt. To reduce computational complexity, a representative node is usually chosen instead of the rebuilding of a new network. However, logic graphs typically have 4–32 normal connections with others, and even more connectivity is also possible considering multicast paths. Therefore, we reconstruct a new network to accurately compute the connection weights between all communities. The weights of the edges between new nodes are the sum of those between nodes in the corresponding two communities. The algorithm performs two phases repeatedly until when no changes occur in the first phase. However, this algorithm automatically determines the number of communities and members in each community in the process. This is inconsistent with our needs because the numbers of chips and cores in the system are immutable. The description of a customized algorithm to handle its limitation is shown in Section IV.

IV. METHODOLOGY

The overview of our approach is depicted in Fig. 6. For training our agent, we leverage the “degenerated” version of DRL proposed in [88], which consists only of a single step size environment. Our model is composed of a single GCN layer to convert the features of logic graphs to a vector, and a fully

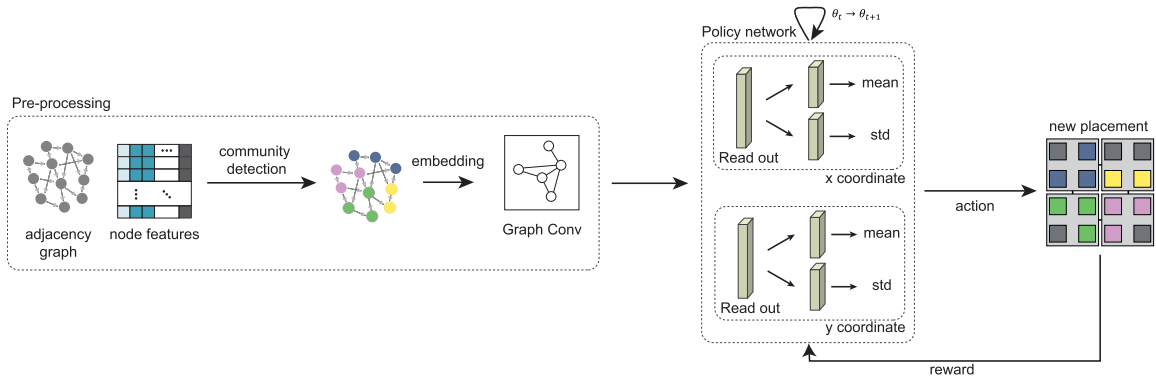


Fig. 6. Overview of our approach. If more than one chip canvas is employed, the netlist graph is clustered first by the proposed community detection algorithm and embedded to the policy network. In each episode, our environment always provides the same initial state to the agent, and it is terminated when one action is taken. The policy network consists of two small neural networks, and they output the probability distributions of x and y coordinates, respectively. The parameters of the policy network are updated with PPO when enough rewards are gathered.

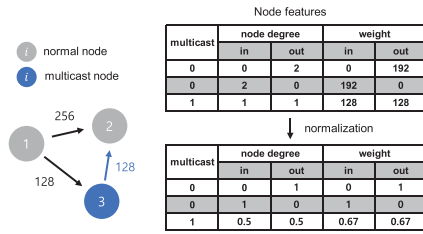


Fig. 7. Example of the node features for a simple network graph. Each core has five components: multicast or not; node degree (in/out); and average weight (in/out). They are normalized by min–max normalization for each dimension.

connected layer is used as a read-out mixing all information of them. ReLU is used as the activation function for the read-out layer, and Tanh is used for the output layer of the policy network to bound the mean to the range of a particular size. In this section, we introduce how to formulate a physical mapping problem with an RL-based approach, followed by a detailed description of the key elements.

A. State Representation

Our state contains information about the netlist graph of a logic graph and its node features. The netlist for a logic graph is represented as an adjacency matrix, and it is normalized to the Laplacian matrix. In node features, there are five components that represent the node properties. An example of node features for a simple network graph is shown in Fig. 7. The first component of the features signifies whether the node is a multicast node or not; 1 represents that the node is a multicast node, whereas 0 represents that it is not. The next two features refer to the node degrees with considering the directions (in/out), respectively. The last two features represent average communication weights computed in the same way. The features except for the first one are normalized by min–max normalization for each dimension. These graph properties are encoded as a vector with a single dimension through a GCN layer, and all components are mixed in the read-out layer. Our environment always provides the same initial state to our agent at the start.

B. Action Representation

We place all of the logic cores in single step size. We use Gaussian distribution and discretize them to handle large dimensional action space. In addition, we exploit two independent small-size fully connected layers to facilitate search in either direction. If n logic cores are mapped into $r \times c$ 2-D mesh array, the policy network outputs $2n$ of mean μ and standard deviation σ . In other words, it outputs four variables for every single logic core. After $2n$ Gaussian distributions are formed, they are clipped into $[-x, x]$ first and discretized again into the number of r and c values at equal intervals. These discrete values represent the probabilities of being mapped to those locations, respectively. If more than one core chooses the same physical nodes, the core with a lower logical index is assigned with the higher ones, and according to their priorities, they are placed into the vacancy node that is the first one founded clockwise with the minimum Manhattan distance to its originally chosen location.

C. Reward Representation

In this work, our goal is to find optimal placements with minimizing routing runtime, power consumption, and avoiding deadlock. Therefore, the reward function needs to consider all of them, simultaneously. The routing performance of NoC is commonly evaluated by their own simulator. However, the long time spent on evaluation using the simulator is not suitable as it will directly affect our agent's overall training time. Thus, a reward function needs to be fast to evaluate; ideally, in a few milliseconds, also, this approximated function must be positively correlated with the real performance of the system. Yang *et al.* [77] found that the power consumption and the packet routing delay are linear with respect to communication cost. Based on their experimental results, we customize the communication cost described in Section III to prevent biased routing in some directions, considering simultaneously reducing the usage area. The newly designed objective function $\mathcal{J}_\theta$ is shown as follows:

$$\mathcal{J}_\theta = \min\{(\max(\text{CC}_{\text{left}}, \text{CC}_{\text{right}}) + \max(\text{CC}_{\text{up}}, \text{CC}_{\text{down}})) \times e^{dl_ratio}\} \quad (11)$$

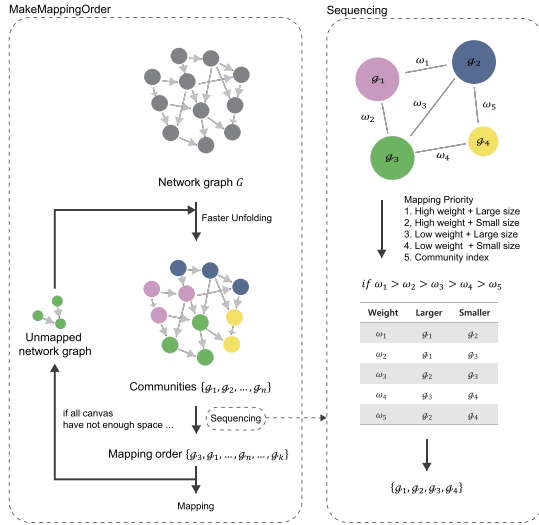


Fig. 8. Key process for the customized community detection algorithm. *MakeMappingOrder* gets the network graph, and it outputs the mapping order by taking two phases. First, the faster unfolding algorithm is performed to detect communities. Second, the *sequencing* function is executed to ordering them. It is repeated until all logic cores are assigned a chip index.

where $CC_{\text{left(right,up,down)}}$ is the communication cost of route, including the left(right,up,down) direction. For example, CC_{left} is increased if A, E, and H in Fig. 2(a) occur, and the route distance of up or down direction is ignored when the communication cost is calculated. After four-partial-communication cost of a new placement is obtained, we only select the larger one between left and right and the one between up and down. dl_ratio in this equation is the ratio of the number of existences of prohibited connection relationships and that of the paths, including the multicast node of the logic graph.

For finding deadlock-free core placement, we take into account this problem as a soft constraint, reducing the number of several routing directions [in our work, we prohibit two connection relationships ⑧ and ⑫ depicted in Fig. 2(b)] that compose deadlock dependence cycle. Exponential is used to do a tradeoff between communication cost and the deadlock penalty. Our agent learns to find an optimal placement that minimizes the objective function.

D. Partitioning With Community Detection Algorithm

In order to reduce the complexity and execution time of mapping cores for multichip systems, we use the faster unfolding algorithm as a graph partitioner, and our purpose is to minimize the traffic of interchip data transmission. As mentioned above, the existing algorithms do not involve controlling the number of clusters and that of members in each one. Based on this, we design a new loop containing the original algorithm to make it consider these components. The recursive process is depicted simply in Fig. 8, and the entire steps of the method that we used are performed as follows. In the beginning phase, vacancy communities that are not covering any nodes are generated corresponding to the number of chips in a physical device; then, the weights of

links in the original logic graph to be grouped are recalculated as the sum of them between any two nodes for excluding the directional attribute. Next, the faster unfolding algorithm is performed for this nondirectional network without any additional conditions in this phase. After the first partitioning network, every two connected communities are grouped as a triplet—weight, $comm_i$, $comm_j$, and they are sorted in descending order according to the first component. The order represents the priority of being placed, and a community with a large number of members is placed first within a group. If the community to be placed is too large, so the vacancy space of every chip is inadequate, previous steps are performed again for this community, which only considers inner links. The subdivided communities are resorted only between them and keep their existing priority in entire communities. This process ends when all communities are placed. After all cores of logic graphs are grouped, the probability distributions, which are the outputs of the policy network, are discretized into the number of nodes in a single chip, and they are only placed on a preallocated chip.

E. Parameter Update

The environment only has a single timestep, which represents a significant difference with the classic MDP process. RL agent once attempts to mapping all cores of logic graph for each episode and only takes a single triplet (s_0, a, r) to update its parameters. Consequently, we only leverage the capability of DRL learning from indirect supervision as a direct nonlinear optimizer. We also use PPO, which is one of state-of-the-art on-policy actor-critic algorithms and optimizes the surrogate loss with a clipped objective, as shown in (12), to update the parameters of our policy network

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{E}_t[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)]. \quad (12)$$

A GCN layer is only responsible for converting the graph information to a vector, so their parameters are not updated during optimization. The entire procedure of our approach is summarized in Algorithm 1.

V. EXPERIMENTS

A. Experiments Details

1) *Dataset and Setup*: Six datasets, two of each with small, medium, and large sizes, are used to evaluate the optimization results of our agent. These datasets are logic graphs for MLP, CNN, and ResNet50. Fig. 9 shows a visualization of our datasets, and the titles represent the specific size of layers. About the first four datasets, 54 and 154 represent the number of logic cores. MLP54 and MLP154 are task graphs with four and six fully connected layers, respectively, and each number of titles represents the size of the layer. In the CNN dataset, the first number represents the size of the input feature map. 6c3 in CNNs means that this layer has 3×3 kernel size and six output channels. fc128 stands for a fully connected layer with 128 neurons. They have multicast nodes for about 20%–35%. We compare our agent with other baseline

Algorithm 1 Reinforcement-Based Physical Mapping With Community Detection

```

1: Initialize communities  $comm_{init}$ 
2: Initialize initial state  $s_{init}$ 
3: Initialize parameters  $\theta$  for the policy network
4: /*pre-process for mapping to multi-chip
   system*/
5:  $comm_{idx} \leftarrow 0$  ▷ the index of community
6:  $idx \leftarrow 0$  ▷ the index of chip
7:  $comm \leftarrow Fast\_Unfolding(comm_{init})$ 
8:  $Sequencing(comm)$  ▷ sort them by connection weights
9: while unplaced community exists do
10:  $comm_i \leftarrow comm[comm_{idx}]$ 
11: while  $num(comm_i) > forall(Chip_{left})$  do
12:  $comm_{new\_i} \leftarrow Fast\_Unfolding(comm_i)$ 
13:  $Sequencing(comm_{new\_i})$ 
14: Insert  $comm_{new\_i}$  into  $comm[comm_{idx}]$ 
15:  $comm_i \leftarrow comm[comm_{idx}]$ 
16: end while
17:
18: while  $num(comm_i) > Chip_{left}[idx]$  do
19:  $idx \leftarrow idx + 1$  ▷ assign the next chip
20: end while
21:  $Chip_{left}[idx] \leftarrow Chip_{left}[idx] - num(comm_i)$ 
22: Assign  $idx$  to members of  $comm_i$ 
23: end while
24: /*physical mapping*/
25:  $r_{best} \leftarrow \infty$ 
26:  $\alpha \leftarrow$  convergence ratio
27:  $n \leftarrow 0$  ▷ terminate process signal
28: while  $n < n_{max}$  do
29:  $t \leftarrow 0$ 
30: while  $t < batch\_size$  do
31: Perform actions  $a_t$  based on policy  $\mu_\theta(s_{init})$ 
32: Recieve reward  $r_t$  by equation (11)
33: if  $r_t < r_{best}$  then
34:  $r_{best} \leftarrow r_t$ 
35: else if  $r_t \leq r_{best} * \alpha$  then
36:  $n \leftarrow n + 1$ 
37: else
38:  $n \leftarrow 0$ 
39: end if
40: Add  $(s_{init}, a_t, r_t)$  into buffer
41:  $t \leftarrow t + 1$ 
42: end while
43: Update  $\theta$  with PPO
44: end while

```

described in Section V-B for these four datasets in a single-chip system, two sequential placement schemes, the random search, and SA. The BIGCNN dataset is a logic graph of a CNN larger than CNN154, and it is composed of more than 600 cores. The last dataset is the workload for a time phase of ResNet50, a well-known deep neural network structure, and consists of more than 1400 cores. For these two datasets, we only evaluate the packet distributions and traffic balances

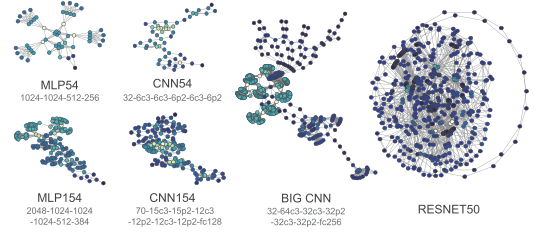


Fig. 9. Visualization of datasets that are used in the experiments. We used six datasets with 50+ to 1400+ cores.

for inner chip and interchip routings of optimized placement. The results are compared to the optimal placements obtained manually, which partitions logic graphs depending on the original neural network structure and places them in order, and our agent without applying community detection algorithm. As over half of cores are multicast nodes and search space is very huge, other schemes are hard to find a valid placement.

For structures and hyperparameters in our approach, we convert the netlist information into features with a size of 32 using a GCN layer and use two small neural networks with 512 readout layers to obtain new placements. Empirically, clipping_range of probability distribution and batch_size have a significant impact on optimal quality, so we set them to 0.1 and 256 for small-size chip canvas and 0.2 and 1024 for larger one. We initialize them with orthogonal, and our learning rate is set to $lr = 0.005$ with $ppo_epoch = 10$ and $ppo_clip = 0.5$. The normalized rewards are clipped into $[-10, 10]$ regardless of chip size. Since our environment only has a single step size, the advantage $\hat{A}_t$ in (12) is replaced with the reward r , and no discount factor is required.

All results are evaluated with a C++-based cycle-accurate routing simulator designed for our multichip many-core system. This simulator simulates the state of all packets running in each cycle of the device. Without loss of generality, we use 2-D mesh topology and dimension order routing for all our results.

2) *Baseline*: Our RL-based agent is evaluated and compared with the following placement methods as a baseline.

- 1) *Sequential Placement*: There are two kinds of sequential placements: *zigzag* and *neighbor*. They are mapping methods of placing logic cores on physical routers in order row by row directly. *Zigzag* always starts to place logic cores from the top left, whereas *neighbor* places them from left and from right alternately depending on whether the row is even or not. *Zigzag* and *neighbor* are denoted by SP_Z and SP_N, respectively.
- 2) *Random Search*: 300 000 placements are randomly generated, and the placement with the minimum routing runtime is selected as a baseline. The random search method is denoted by RS.
- 3) *SA*: It is a powerful but slow optimization method, which approximates the global optima in a large search space. The detailed procedure is described in Algorithm 2. In this experiment, we randomly initialize the input placement and use the same cost function as our RL-based agent. We run 32 times (over one million placements per time) for each dataset with different

Algorithm 2 SA for Mapping

```

1: Randomly generate initial core placement  $\mathcal{P}_{init}$ 
2: Initialize temperatures  $T_{init}$  and  $T_{threshold}$ 
3: Initialize inner loop number  $n_{loop}$  and cooldown factor  $\alpha$ 
4:  $\mathcal{P} \leftarrow \mathcal{P}_{init}$ 
5:  $cost \leftarrow compute\_cost(\mathcal{P}_{init})$ 
6:  $T \leftarrow T_{init}$ 
7: while  $T > T_{threshold}$  do
8:   /*Randomly selected two nodes are
   exchanged their locations and use the
   same cost function with ours*/
9:   for  $loop = 1 : n_{loop}$  do
10:     $cost \leftarrow compute\_cost(\mathcal{P})$ 
11:     $\mathcal{P}_{new} \leftarrow switch\_node(\mathcal{P})$ 
12:     $cost_{new} \leftarrow compute\_cost(\mathcal{P}_{new})$ 
13:     $\Delta C \leftarrow cost_{new} - cost$ 
14:    if  $\Delta C < 0$  then
15:       $\mathcal{P} \leftarrow \mathcal{P}_{new}$ 
16:       $cost \leftarrow cost_{new}$ 
17:    else if  $e^{-\frac{\Delta C}{T}} > random(0, 1)$  then
18:       $\mathcal{P} \leftarrow \mathcal{P}_{new}$ 
19:       $cost \leftarrow cost_{new}$ 
20:    end if
21:  end for
22:   $T \leftarrow \alpha T$ 
23: end while

```

hyperparameters, such as min and max temperatures, temperature attenuation coefficient, and the number of iterations at each epoch. The placement with minimum routing runtime is selected as a baseline. The SA method is denoted by SA.

- 4) *RL Agent*: Many placements can be performed in various routing runtime although they have similar communication costs. With taking this notice, we assume that our agent is converged when 1000 consecutive placements' rewards are within 1%–5% of the difference from the minimum reward obtained in training; 100 new placements are generated with the trained model, and the one with the minimum routing runtime is compared.

B. Deadlock Ratio

One of our main purposes is to find a deadlock-free placement. Therefore, before proceeding with optimization, we simply measure the effect of our deadlock constraint for the first four datasets. We compare the deadlock ratio by considering two different routing priorities, which refers to the order of data transmission by their going directions in a multicast router. The condition that multicast routing has the highest (lowest) priority is notated by HP (LP). The first two 54-core graphs are placed into different sizes of chip canvas and compared. The evaluated placements are obtained by RS and two RL-based agents that one of them is considering the deadlock constraint and the other, which is denoted by RL_N, is not. 100k and 1000 placements are generated by RS and two

TABLE I
DEADLOCK INCIDENCE RATIO WITH HP MULTICAST ROUTING

Dataset (size)	MLP54 (8×8)	CNN54 (8×8)	MLP54 (16×16)	CNN54 (16×16)	MLP154 (16×16)	CNN154 (16×16)
RS	48.5 %	40.0 %	8.2 %	14.8 %	100.0 %	100.0 %
RL_N	0.0 %	0.0 %	0.0 %	0.0 %	78.7 %	84.3 %
RL	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %

TABLE II
DEADLOCK INCIDENCE RATIO WITH LP MULTICAST ROUTING

Dataset (size)	MLP54 (8×8)	CNN54 (8×8)	MLP54 (16×16)	CNN54 (16×16)	MLP154 (16×16)	CNN154 (16×16)
RS	43.1 %	33.1 %	7.5 %	14.0 %	100.0 %	100.0 %
RL_N	0.0 %	0.0 %	0.0 %	0.0 %	41.3 %	38.7 %
RL	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %	0.0 %

agents with a 5% convergence ratio. The results are described in Tables I and II.

As aforementioned, we need to avoid some unexpected routing directions that are not used in vanilla dimension order routing to find a deadlock-free placement. For the results of two 54-core graphs with RS, there are significant differences depending on the size of the chip canvas. Since the number of physical nodes decreases, the prohibited connection relationships have more easily emerged. In addition, the more routing paths share into the same link, the easier it is to form resource-dependent cycles. When the size of a chip canvas is increased to 16×16 for them, packets are allocated sparser, and the average number of packets transmitted of each node is reduced naturally. Consequently, the deadlock ratio is also decreased, but it is not a fundamental solution. As shown in the results, the deadlock-free placements of these two small datasets can be found by our agent without applying the constraint in both sizes of chip canvas. It seems that their graphs are relatively uncomplicated and have fewer multicast nodes, so most of the logic cores are placed in regional with the minimum hop distance after optimization.

However, for two 154-core graphs, even if only about 60% of the physical nodes are theoretically exploited, a deadlock occurs with a probability, which is close to 100% or 100% with RS. The original neural network models of these two datasets are larger and more complex; thus, it leads to a high proportion of multicast nodes. The more multicast nodes are adopted, the more prohibited connection relationships are come up within a limited-size chip, and the higher deadlock incidence ratio has emerged. Unlike two small datasets, it still occurs with a high probability after the optimization without the deadlock penalty. After concerning the constraint, our agent is enough to find deadlock-free placements. In addition, there is a significant difference between the results with the two routing priorities. The reason is that the deadlock occurs unconditionally when a multicast chain, which is a long routing path formed by multiple multicast nodes, passes a link between two adjacent nodes more than once. However, this assumption is not applied when the multicast routing has the lowest priority. The result described in Tables I and II

TABLE III
COMPARISON RESULTS OF OUR AGENT TO OTHER BASELINES

Dataset (size)	Method	Latency-HP (<i>cycles</i>)	Latency-LP (<i>cycles</i>)	Power Consumption (<i>mW</i>)	Area (%)	Bandwidth Req. (<i>packets</i>)
MLP54 (8×8)	SP_Z	2175	1355	16.15	84.38	1024
	SP_N	1977	1677	16.18	87.5	1024
	RS	832	822	19.35	100.0	576
	SA	891	805	13.66	87.5	640
	RL	705	658	10.46	84.38	512
CNN54 (8×8)	SP_Z	888	869	10.10	84.38	702
	SP_N	1355	842	7.88	92.19	840
	RS	581	590	12.89	100.0	471
	SA	475	563	4.92	85.94	432
	RL	427	435	4.96	85.04	360
MLP54 (16×16)	SP_Z	2818	2057	22.64	21.09	2048
	SP_N	3326	2057	32.12	21.09	2048
	RS	765	696	32.98	89.75	512
	SA	711	897	13.50	21.09	512
	RL	706	626	10.71	21.09	512
CNN54 (16×16)	SP_Z	1874	894	14.94	21.09	864
	SP_N	1992	842	8.13	21.09	840
	RS	457	450	24.88	94.14	432
	SA	460	437	4.99	27.34	432
	RL	374	387	4.93	22.66	324
MLP154 (16×16)	SP_Z	3500	2102	75.52	60.15	1792
	SP_N	4848	2564	84.63	60.15	2432
	RS	—	—	—	—	—
	SA	1497	1410	46.47	69.92	736
	RL	1280	1074	42.15	64.06	768
CNN154 (16×16)	SP_Z	6395	2178	89.79	60.15	1816
	SP_N	5126	2425	79.13	60.15	2160
	RS	2965	2590	138.29	99.15	1129
	SA	1352	1230	31.84	68.75	920
	RL	852	928	31.55	66.41	632

shows that our approach is effective to find a deadlock-free placement.

C. Placement Optimization for Single-Chip System Analysis

We first optimize each of the two small and two medium datasets for a single-chip system and compare them to other baselines in terms of five categories containing two latencies under HP and LP mentioned above. To make comparisons fair, we use power consumption, which has a linear relationship with communication cost and our objective function without the deadlock penalty, as a comparison option to evaluate the degree of optimization. Area and bandwidth requirements represent the number of cores used at least once and the maximum number of packets transmitted, which is calculated by our simulator in a single channel, respectively. Two 54-core graphs are mapped to a small chip canvas (8×8) and a large chip canvas (16×16), and the results are described in Table III. Since deadlock occurs in all placements generated by RS, the RS line of the MLP154 is omitted.

The agent is converged after taking 15k–70k placements on average according to the canvas size or complexity of logic graphs. Considering all the datasets, on the geometric

average, our agent reduces 70.8%, 74.9%, 27.8%, and 16.9% with HP and reduces 55.7%, 58.8%, 26.9%, and 21.8% with LP compared with SP_Z, SP_N, RS, and SA, respectively. Regarding power consumption, our agent shrinks 50.5%, 48.1%, 67.2%, and 10.9% compared with other baselines, respectively. For two CNN datasets, SA and our agent result in similar sized power consumption, and it means that the degrees of optimizations are also mostly the same. However, the routing runtime of our agent is smaller than SA. This is because our maximum bandwidth requirement is smaller than that of SA. It suggests that the low-congested core placement is found easily with maintaining similar power consumption simultaneously when the policy network is converged.

Unlike the optimal results of CNN datasets, our agent significantly outperforms SA with MLP datasets. In general, larger and more complex models have a larger search space and make the optimization space more uneven, making it more difficult for explorers to find an optimal solution. Since fully connected layers are denser than convolutional layers, the logic graph of MLP is naturally more complex than CNN's. SP_Z and SP_N have the smallest area usage on average, but there are a lot of routing paths that overlap the same channel, which leads to an increase in routing time. These results show that our

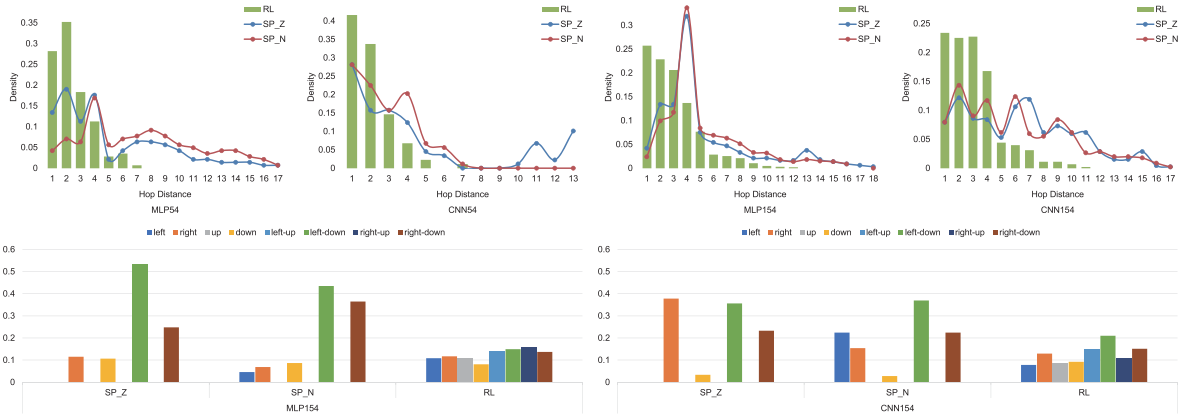


Fig. 10. Top: distributions of hop distances of core placements placed by two SP methods and our agent. Down: distributions of connection relationships of core placements placed by two SP methods and our agent.

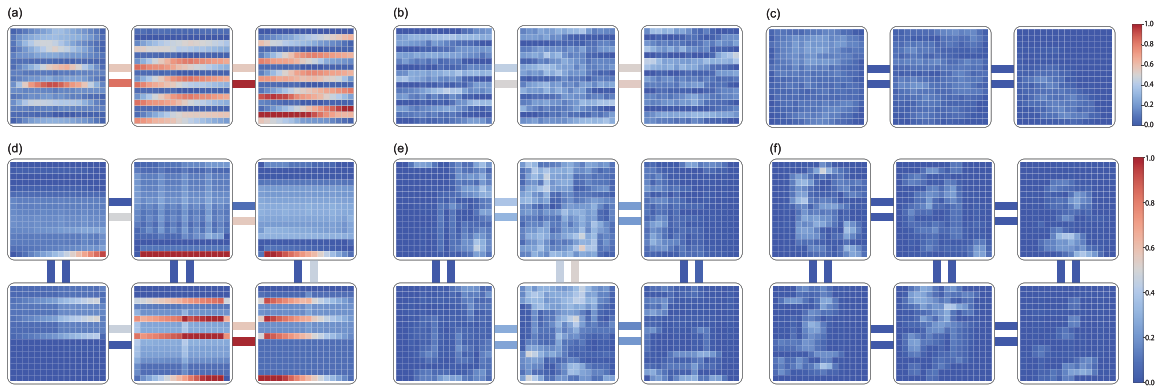


Fig. 11. Visualization of the number of packets transmitted on each core in a single time phase. (a) and (d) are placed by EP, (b) and (e) are placed by the RL, and (c) and (f) are placed by CM-RL. Inner chip and interchip traffics are normalized, respectively.

approach has the advantage of finding the optimal deadlock-free core placement with considering a tradeoff between these options that are used to evaluate the routing performance of NoC.

Fig. 10 shows more insights of core placements generated by our agent. They depict distributions of hop distances of all connections containing multicast routing and eight connection relation statistics when the four datasets are placed on 16×16 chip canvas. The average hop distances in each dataset are reduced by $2.13\times$, $2.23\times$, $1.78\times$, and $2.13\times$, respectively, compared to SP_Z, and reduced $3.16\times$, $1.39\times$, $1.83\times$, and $2.00\times$, respectively, compared to SP_N. In addition, in two sequential placement schemes, only four and five connection relationships are used, whereas all connection relationships are adopted in the balance after optimization with our agent. These results indicate that the cores with connections to each other are placed in close proximity, and all packets are routed in a well-coordinated routing direction in a minimal area; thus, the overall routing runtime and the power consumption are reduced implicitly.

D. Placement Optimization for Multichip System Analysis

We also validate the effectiveness of our agent on the larger datasets containing extremely large search space. BIGCNN and ResNet50 are placed on a multichip system of 3 and 6, respectively. These datasets cannot use two sequential placement schemes that place the cores according to the index

of them because of deadlock caused by a large percentage of multicast nodes. Therefore, we compare the placement provided by our agent to that of human experts, which is denoted by EP, and place the partitioned groups in order with taking account into the position of multicast nodes. In addition, in order to evaluate the effectiveness of applying the customized community detection algorithm, we optimize the placements with two versions of agents. One is our vanilla agent with a weighted objective function, and the other, which is denoted by CM-RL, is to perform physical mapping with a partitioned logic graph by the community detection algorithm. We apply the weighted calculation when the communication cost of the interchip route is computed to make our vanilla agent clarify inner chip and interchip routings. We use the weights of 1, 3, 5, 10, and 15 and choose a placement with the lowest nonweighted communication cost.

Fig. 11 shows the communication traffic of two large datasets obtained by our two agents and EP. The color of each core represents how many packets are transmitted by each node. We normalize for inner chip and interchip routings, respectively. For the EP [see Fig. 11(a) and (d)], there is enormously biased communication traffic both of inner chip and interchip routings. The average traffic of the placements found by our vanilla agent [see Fig. 11(b) and (e)] are clearly reduced than that of one obtained by EP, but it still does not seem to be able to distinguish well between inner chip and interchip routings. However, after applying a community

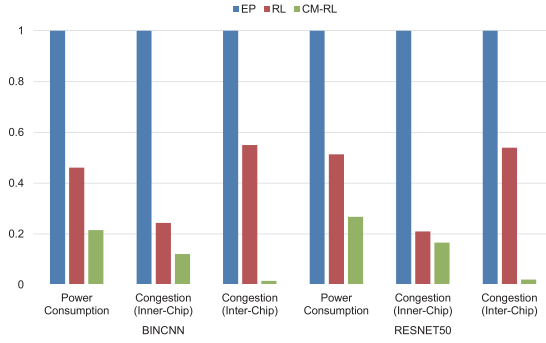


Fig. 12. Power consumption and the maximum requirements of bandwidth of inner chip and interchip routings of different placement methods. All the values are normalized to EP.

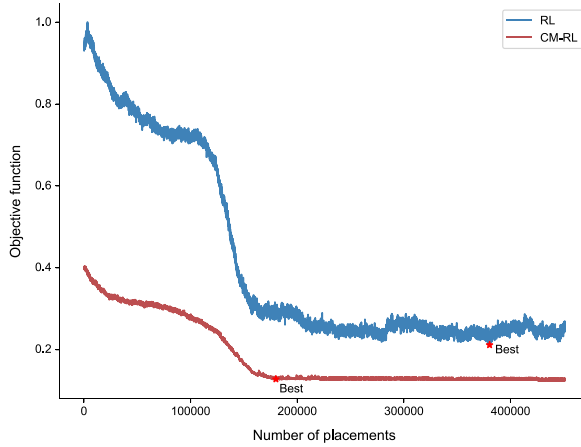


Fig. 13. Convergence plots for training a policy network with the weighted objective function versus with the customized community detection algorithm.

detection algorithm, interchip traffic is significantly reduced than that of other methods. It is because consideration of interchip routing is not required when the agent is being trained. The community detection algorithm makes physical mapping problems easy and reduces the time to find an optimal placement.

Fig. 12 shows the more intuitive comparison between them, and our agent with community detection significantly outperforms the others. After integrating a community detection algorithm to our agent, not only the degree of optimization is improved but also the number of samples required to converge is decreased. It means that the customized algorithm decreases the complexity of the action space of each logic core in a multichip system, and its compressed action space facilitates finding the optimal placement.

Our agent takes about 150k and 380k samples without the customized community detection algorithm to converge these two large datasets, whereas only 80k and 180k placements are required with CM-RL. Fig. 13 shows the learning curves of RL and CM-RL when the netlist of ResNet50 is mapped. When the customized community detection algorithm is not adopted, the learning curve looks like convergence after taking about 180k samples. However, not only it is still very unstable but also the optimal placement is found at taking about 380k samples. Otherwise, the learning curve of CM-RL is converged and finds its optimal placement after only taking

about 180k samples. In addition, the optimal placement of CM-RL significantly outperforms that of RL. As the results can be seen from the above, our agent, which considers the deadlock penalty, has good performances for various sizes of datasets, and the results conspicuously outperform the placements found by others.

VI. CONCLUSION

In this work, an RL-based core placement optimization method both for single-chip and multichip systems is proposed, considering application constraints, such as deadlock problems caused by multicast routing. The carefully designed structure of the policy network and environment configuration, which is always providing the same state to the agent and terminates the episode by taking actions at once, significantly reduces the training time. However, depending on the size and the complexity of logic graphs, it still costs minutes to hours for training to converge. In order to further reduce the training time, we can pretrain GCN layers to obtain the features of different netlists and use them as an encoder component of the policy network. For physical mapping to large-scale multichip systems, performing a customized community detection algorithm as a preprocess provides significant improvements in reducing the maximum bandwidth requirement of interchip routing. Our proposed method shows good performance both for single-chip and multichip systems; also, deadlock can be avoided by applying an exponential penalty in the objective function. In addition, we can prevent other routing problems easily by considering their impacts as soft constraints, such as deadlock.

REFERENCES

- [1] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Dec. 2015, pp. 1026–1034.
- [2] Y. Wang and M. Kosinski, "Deep neural networks are more accurate than humans at detecting sexual orientation from facial images," *J. Personality Social Psychol.*, vol. 114, no. 2, p. 246, 2018.
- [3] Y. Zhang, M. Pezeshki, P. Brakel, S. Zhang, C. L. Y. Bengio, and A. Courville, "Towards end-to-end speech recognition with deep convolutional neural networks," 2017, *arXiv:1701.02720*. [Online]. Available: <http://arxiv.org/abs/1701.02720>
- [4] O. Vinyals *et al.*, "Grandmaster level in StarCraft ii using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.
- [5] D. Silver *et al.*, "Mastering the game of go without human knowledge," *Nature*, vol. 550, no. 7676, pp. 354–359, 2017.
- [6] V. Mnih *et al.*, "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*. [Online]. Available: <http://arxiv.org/abs/1312.5602>
- [7] J. Hwangbo *et al.*, "Learning agile and dynamic motor skills for legged robots," *Sci. Robot.*, vol. 4, no. 26, Jan. 2019, Art. no. eaau5872.
- [8] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 27, 2014, pp. 3104–3112.
- [9] B. R. Kiran *et al.*, "Deep reinforcement learning for autonomous driving: A survey," 2020, *arXiv:2002.00444*. [Online]. Available: <http://arxiv.org/abs/2002.00444>
- [10] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.
- [11] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symp. Operating Syst. Design Implement. (OSDI)*, 2016, pp. 265–283.
- [12] A. Paszke *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 8026–8037.

- [13] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 1–12.
- [14] D. Shin, J. Lee, J. Lee, and H.-J. Yoo, "14.2 DNPu: An 8.1TOPS/W reconfigurable CNN-RNN processor for general-purpose deep neural networks," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2017, pp. 240–241.
- [15] Y. H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.
- [16] F. Akopyan *et al.*, "TrueNorth: Design and tool flow of a 65 mW 1 million neuron programmable neurosynaptic chip," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 34, no. 10, pp. 1537–1557, Oct. 2015.
- [17] S. B. Furber *et al.*, "Overview of the SpiNNaker system architecture," *IEEE Trans. Comput.*, vol. 62, no. 12, pp. 2454–2467, Dec. 2013.
- [18] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan. 2018.
- [19] B. V. Benjamin *et al.*, "Neurogrid: A mixed-analog-digital multichip system for large-scale neural simulations," *Proc. IEEE*, vol. 102, no. 5, pp. 699–716, May 2014.
- [20] D. Xie, F. Li, B. Yao, G. Li, L. Zhou, and M. Guo, "Simba: Efficient in-memory spatial analytics," in *Proc. Int. Conf. Manage. Data*, Jun. 2016, pp. 1071–1085.
- [21] J. Pei *et al.*, "Towards artificial general intelligence with hybrid Tianjic chip architecture," *Nature*, vol. 572, no. 7767, pp. 106–111, Aug. 2019.
- [22] S. K. Esser *et al.*, "Convolutional networks for fast, energy-efficient neuromorphic computing," *Proc. Nat. Acad. Sci. USA*, vol. 113, pp. 11441–11446, Aug. 2016.
- [23] A. Boni, A. Pierazzi, and D. Vecchi, "LVDS I/O interface for Gb/s-per-pin operation in 0.35- μ m CMOS," *IEEE J. Solid-State Circuits*, vol. 36, no. 4, pp. 706–711, Apr. 2001.
- [24] T. Beukema *et al.*, "A 6.4-Gb/s CMOS SerDes core with feed-forward and decision-feedback equalization," *IEEE J. Solid-State Circuits*, vol. 40, no. 12, pp. 2633–2645, Dec. 2005.
- [25] U. Y. Ogras, J. Hu, and R. Marculescu, "Key research problems in NoC design: A holistic perspective," in *Proc. 3rd IEEE/ACM/FIP Int. Conf. Hardw./Softw. Codesign Syst. Synth. (CODES+ISSS)*, Sep. 2005, pp. 69–74.
- [26] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," 2016, *arXiv:1611.09940*. [Online]. Available: <http://arxiv.org/abs/1611.09940>
- [27] Q. Ma, S. Ge, D. He, D. Thaker, and I. Drori, "Combinatorial optimization by graph pointer networks and hierarchical reinforcement learning," 2019, *arXiv:1911.04936*. [Online]. Available: <http://arxiv.org/abs/1911.04936>
- [28] T. Barrett, W. Clements, J. Foerster, and A. Lvovsky, "Exploratory combinatorial optimization with reinforcement learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 4, 2020, pp. 3243–3250.
- [29] K. Feng, Q. Wang, X. Li, and C. Wen, "Deep reinforcement learning based intelligent reflecting surface optimization for MISO communication systems," *IEEE Wireless Commun. Lett.*, vol. 9, no. 5, pp. 745–749, May 2020.
- [30] Q. Cappart, T. Moisan, L.-M. Rousseau, I. Prémont-Schwarz, and A. Cire, "Combining reinforcement learning and constraint programming for combinatorial optimization," 2020, *arXiv:2006.01610*. [Online]. Available: <http://arxiv.org/abs/2006.01610>
- [31] N. Mazyavkina, S. Sviridov, S. Ivanov, and E. Burnaev, "Reinforcement learning for combinatorial optimization: A survey," *Comput. Oper. Res.*, vol. 134, Oct. 2021, Art. no. 105400.
- [32] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*. [Online]. Available: <http://arxiv.org/abs/1707.06347>
- [33] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [34] A. Bender, "MILP based task mapping for heterogeneous multiprocessor systems," in *Proc. EURO-DAC Eur. Design Autom. Conf. With EURO-VHDL Exhib.*, Sep. 1996, pp. 190–197.
- [35] C.-E. Rhee, H.-Y. Jeong, and S. Ha, "Many-to-many core-switch mapping in 2-D mesh NoC architectures," in *Proc. IEEE Int. Conf. Comput. Design, VLSI Comput. Processors (ICCD)*, Oct. 2004, pp. 438–443.
- [36] C. Ostler and K. S. Chatha, "An ILP formulation for system-level application mapping on network processor architectures," in *Proc. Design, Autom. Test Eur. Conf. Exhib.*, Apr. 2007, pp. 1–6.
- [37] T. Lei and S. Kumar, "A two-step genetic algorithm for mapping task graphs to a network on chip architecture," in *Proc. Euromicro Symp. Digit. Syst. Design*, Sep. 2003, pp. 180–187.
- [38] W. Zhou, Y. Zhang, and Z. Mao, "An application specific NoC mapping for optimized delay," in *Proc. Int. Conf. Design Test Integr. Syst. Nanosc. Technol. (DTIS)*, Sep. 2006, pp. 184–188.
- [39] J. Wang, Y. Li, S. Chai, and Q. Peng, "Bandwidth-aware application mapping for NoC-based MPSoCs," *J. Comput. Inf. Syst.*, vol. 7, no. 1, pp. 152–159, 2011.
- [40] Y. Liu, Y. Ruan, Z. Lai, and W. Jing, "Energy and thermal aware mapping for mesh-based NoC architectures using multi-objective ant colony algorithm," in *Proc. 3rd Int. Conf. Comput. Res. Develop.*, Mar. 2011, pp. 407–411.
- [41] A. Benyamina, P. Boulet, A. Aroui, S. Eltar, and K. Dellal, "Mapping real time applications on NoC architecture with hybrid multi-objective algorithm," in *Proc. Int. Conf. Metaheuristics Nature Inspired, Comput.*, Oct. 2010, pp. 1–10.
- [42] P. K. Sahu, P. Venkatesh, S. Gollapalli, and S. Chattopadhyay, "Application mapping onto mesh structured network-on-chip using particle swarm optimization," in *Proc. IEEE Comput. Soc. Annu. Symp. (VLSI)*, Jul. 2011, pp. 335–336.
- [43] C. Ma, Q. Zhao, G. Li, L. Deng, and G. Wang, "A deadlock-free physical mapping method on the many-core neural network chip," *Neurocomputing*, vol. 401, pp. 327–337, Aug. 2020.
- [44] J. Huang, C. Buckl, A. Raabe, and A. Knoll, "Energy-aware task allocation for network-on-chip based heterogeneous multiprocessor systems," in *Proc. 19th Int. Euromicro Conf. Parallel, Distrib. Netw.-Based Process.*, Feb. 2011, pp. 447–454.
- [45] S. Murali and G. De Micheli, "Bandwidth-constrained mapping of cores onto NoC architectures," in *Proc. Design, Autom. Test Eur. Conf. Exhib.*, Feb. 2004, pp. 896–901.
- [46] M. Janidarmian, A. Khademzadeh, and M. Tavanpour, "Onyx: A new heuristic bandwidth-constrained mapping of cores onto tile-based network on chip," *IEICE Electron. Exp.*, vol. 6, no. 1, pp. 1–7, 2009.
- [47] P. K. Sahu and S. Chattopadhyay, "A survey on application mapping strategies for network-on-chip design," *J. Syst. Archit.*, vol. 59, no. 1, pp. 60–76, 2013.
- [48] W. Amin *et al.*, "Performance evaluation of application mapping approaches for network-on-chip designs," *IEEE Access*, vol. 8, pp. 63607–63631, 2020.
- [49] P. Chi *et al.*, "Prime: A novel processing-in-memory architecture for neural network computation in ReRAM-based main memory," *ACM SIGARCH Comput. Archit. News*, vol. 44, no. 3, pp. 27–39, Jun. 2016.
- [50] L. Deng *et al.*, "SemiMap: A semi-folded convolution mapping for speed-overhead balance on crossbars," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 39, no. 1, pp. 117–130, Jan. 2020.
- [51] N. Viswanathan, G.-J. Nam, C. J. Alpert, P. Villarrubia, H. Ren, and C. Chu, "RQL: Global placement via relaxed quadratic spreading and linearization," in *Proc. 44th ACM/IEEE Design Autom. Conf.*, Jun. 2007, pp. 453–458.
- [52] X. Jin, "Parallel simulation of neural networks on spinnaker universal neuromorphic hardware," Ph.D. dissertation, School Comput. Sci., Univ. Manchester United Kingdom, Manchester, U.K., 2010.
- [53] N. Wu, L. Deng, G. Li, and Y. Xie, "Core placement optimization for multi-chip many-core neural network systems with reinforcement learning," *ACM Trans. Design Autom. Electron. Syst.*, vol. 26, no. 2, pp. 1–27, Feb. 2021.
- [54] W. Zhang, C. Wu, H. Zhong, Y. Li, and L. Wang, "Prediction of undrained shear strength using extreme gradient boosting and random forest based on Bayesian optimization," *Geosci. Frontiers*, vol. 12, no. 1, pp. 469–477, Jan. 2021.
- [55] W. G. Zhang, H. R. Li, C. Z. Wu, Y. Q. Li, Z. Q. Liu, and H. L. Liu, "Soft computing approach for prediction of surface settlement induced by Earth pressure balance shield tunneling," *Underground Space*, vol. 6, no. 4, pp. 353–363, Aug. 2021.
- [56] W. Zhang *et al.*, "State-of-the-art review of soft computing applications in underground excavations," *Geosci. Frontiers*, vol. 11, no. 4, pp. 1095–1106, Jul. 2020.
- [57] W. Zhang, H. Li, Y. Li, H. Liu, Y. Chen, and X. Ding, "Application of deep learning algorithms in geotechnical engineering: A short critical review," *Artif. Intell. Rev.*, pp. 1–41, Feb. 2021.
- [58] A. T. C. Goh, R. H. Zhang, W. Wang, L. Wang, H. L. Liu, and W. G. Zhang, "Numerical study of the effects of groundwater draw-down on ground settlement for excavation in residual soils," *Acta Geotechnica*, vol. 15, no. 5, pp. 1259–1272, May 2020.

- [59] D. Vashisht *et al.*, "Placement in integrated circuits using cyclic reinforcement learning and simulated annealing," 2020, *arXiv:2011.07577*. [Online]. Available: <http://arxiv.org/abs/2011.07577>
- [60] H. Liao, W. Zhang, X. Dong, B. Poczos, K. Shimada, and L. B. Kara, "A deep reinforcement learning approach for global routing," *J. Mech. Design*, vol. 142, no. 6, Jun. 2020, Art. no. 061701.
- [61] A. Goldie and A. Mirhoseini, "Placement optimization with deep reinforcement learning," in *Proc. Int. Symp. Phys. Design*, Mar. 2020, pp. 3–7.
- [62] A. Mirhoseini *et al.*, "Chip placement with deep reinforcement learning," 2020, *arXiv:2004.10746*. [Online]. Available: <http://arxiv.org/abs/2004.10746>
- [63] H. Zheng and A. Louri, "An energy-efficient network-on-chip design using reinforcement learning," in *Proc. 56th Annu. Design Autom. Conf.*, Jun. 2019, pp. 1–6.
- [64] K. Wang, A. Louri, A. Karanth, and R. Bunesco, "High-performance, energy-efficient, fault-tolerant network-on-chip design using reinforcement learning," in *Proc. Design, Autom. Test Eur. Conf. Exhib. (DATE)*, Mar. 2019, pp. 1166–1171.
- [65] K. Shahookar and P. Mazumder, "VLSI cell placement techniques," *ACM Comput. Surv.*, vol. 23, no. 2, pp. 143–220, 1991.
- [66] A. Nair *et al.*, "Massively parallel methods for deep reinforcement learning," 2015, *arXiv:1507.04296*. [Online]. Available: <http://arxiv.org/abs/1507.04296>
- [67] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 229–256, 1992.
- [68] A. Mirhoseini *et al.*, "Device placement optimization with reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 2430–2439.
- [69] A. Mirhoseini, A. Goldie, H. Pham, B. Steiner, Q. V. Le, and J. Dean, "A hierarchical model for device placement," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–11.
- [70] Y. Gao, L. Chen, and B. Li, "Spotlight: Optimizing device placement for training deep neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1676–1684.
- [71] Y. Gao, L. Chen, and B. Li, "Post: Device placement with cross-entropy minimization and proximal policy optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 9971–9980.
- [72] R. Addanki, S. B. Venkatakrishnan, S. Gupta, H. Mao, and M. Alizadeh, "Placeto: Learning generalizable device placement algorithms for distributed machine learning," 2019, *arXiv:1906.08879*. [Online]. Available: <http://arxiv.org/abs/1906.08879>
- [73] B. Jeon *et al.*, "Baechi: Fast device placement of machine learning graphs," in *Proc. 11th ACM Symp. Cloud Comput.*, Oct. 2020, pp. 416–430.
- [74] N. Jiang, G. Michelogiannakis, D. Becker, B. Towles, and W. J. Dally, *Booksim 2.0 User's Guide*. Stanford, CA, USA: Stanford Univ., 2010.
- [75] V. Catania, A. Mineo, S. Monteleone, M. Palesi, and D. Patti, "Noxim: An open, extensible and cycle-accurate network on chip simulator," in *Proc. IEEE 26th Int. Conf. Appl.-Specific Syst., Archit. Processors (ASAP)*, Jul. 2015, pp. 162–163.
- [76] M. Lis, K. S. Shim, M. H. Cho, P. Ren, O. Khan, and S. Devadas, "DARSIM: A parallel cycle-level NoC simulator," in *Proc. 6th Annu. Workshop Modeling, Benchmarking Simulation (MoBS)*, 2010.
- [77] B. Yang, T. C. Xu, T. Santti, and J. Plosila, "Tree-model based mapping for energy-efficient and low-latency network-on-chip," in *Proc. 13th IEEE Symp. Design Diag. Electron. Circuits Syst.*, Apr. 2010, pp. 189–192.
- [78] R. Bellman, "A Markovian decision process," *Indiana Univ. Math. J.*, vol. 6, no. 4, pp. 679–684, 1957.
- [79] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [80] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, 1992.
- [81] R. S. Sutton, D. A. McAllester, S. P. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. NIPS*, vol. 99, 1999, pp. 1057–1063.
- [82] M. E. J. Newman, "Analysis of weighted networks," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 70, no. 5, Nov. 2004, Art. no. 056131.
- [83] M. Girvan and M. E. J. Newman, "Community structure in social and biological networks," *Proc. Nat. Acad. Sci. USA*, vol. 99, no. 12, pp. 7821–7826, Apr. 2002.
- [84] J. Xiang *et al.*, "Comparing local modularity optimization for detecting communities in networks," *Int. J. Modern Phys. C*, vol. 28, no. 6, Jun. 2017, Art. no. 1750084.
- [85] B. Saoud and A. Moussaoui, "Node similarity and modularity for finding communities in networks," *Phys. A, Stat. Mech. Appl.*, vol. 492, pp. 1958–1966, Feb. 2018.
- [86] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *J. Stat. Mech., Theory Exp.*, vol. 2008, no. 10, Oct. 2008, Art. no. P10008.
- [87] V. A. Traag, "Faster unfolding of communities: Speeding up the Louvain algorithm," *Phys. Rev. E, Stat. Phys. Plasmas Fluids Relat. Interdiscip. Top.*, vol. 92, no. 3, Sep. 2015, Art. no. 032801.
- [88] J. Viquerat, J. Rabault, A. Kuhnle, H. Ghraieb, A. Larcher, and E. Hachem, "Direct shape optimization through deep reinforcement learning," *J. Comput. Phys.*, vol. 428, Mar. 2021, Art. no. 110080.



Wooshik Myung received the B.E. degree in precision instrument from Tsinghua University, Beijing, China, in 2017, and the M.S. degree from the Center for Brain Inspired Computing Research (CBICR), Department of Precision Instrument, Tsinghua University, in 2021.

He is currently an ML Research Engineer with Makinarocks, Seoul, South Korea. His current research interests include optimization algorithms, reinforcement learning, neuromorphic computing, and network-on-chip systems.



Donghyun Lee received the B.E. degree in mechanical engineering from Yonsei University, Seoul, South Korea, in 2018, and the M.S. degree from the Center for Brain Inspired Computing Research (CBICR), Department of Precision Instrument, Tsinghua University, Beijing, China, in 2021.

He is currently an AI Researcher with the Department of Materials Science and Engineering, Ulsan National Institute of Science and Technology, Ulsan, South Korea. His current research interests include efficient machine learning algorithms, compressing deep learning models, and brain-inspired computing.



Chenhang Song was born in Shanxi, China, in 1998. He received the B.Eng. degree in measurement and control instrument from the Department of Precision Instrument, Tsinghua University, Beijing, China, in 2020, where he is currently pursuing the M.Eng. degree in electronic information engineering.

His area of specialization is in the mapping of artificial neural networks for multichip many-core systems.



Guanrui Wang received the B.E. degree in electronic engineering from Jilin University, Jilin, China, in 2016. He is currently pursuing the Ph.D. degree with the Center for Brain Inspired Computing Research (CBICR), Department of Precision Instrument, Tsinghua University, Beijing, China.

His current research interests include brain-inspired computing, neuromorphic chips, computer architecture, and network-on-chip systems.



Cheng Ma received the B.E. and M.E. degrees in mechanical engineering from Beijing Institute of Technology, Beijing, China, in 1993 and 1996, respectively, and the Ph.D. degree from Tsinghua University, Beijing, in 2000.

He is currently an Associate Professor with the Center for Brain Inspired Computing Research (CBICR), Department of Precision Instrument, Tsinghua University. He has authored or coauthored more than 100 journal articles and conference papers. His current research interests include brain-inspired computing, neuromorphic computing, digital signal processing, and intelligent instruments.